ASG Instruction Fine Tuning Dataset Requirements and Design

Date: 10/29/2025

Author: Dr. Ionut Cardei

NOTES

1. The template language grammar was added at the end of the document.
2. As of 09/2025 the ASG CTI neo4j graph DB has a schema that does not fully match the ATT&CK schema, and the structure of the other CTI source documents. CTI domain experts writing generation templates need to weigh in on the concepts, relationships and properties needed by templates. See more in Section 3.

Table of Contents

1. Introduction

For IFT to be successful, proper coverage of CTI concepts ad relationships are key. Templates must be written with meaningful concepts with expressive relationships that target at least one of these two fundamental objectives:

- Business centric: focused on the essential CTI LLM use cases that one expects to see in a real deployment or that have been seen in testing or deployment underperform.
- Test centric: focused on maximizing the CTI bench test scores.

Basic templates will be needed to describe concept properties and their direct relationships to other concepts. These are needed for information recall, mostly.

More sophisticated templates are needed to teach the LLM how to reason for non-trivial questions that span multiple concepts, longer paths in the concept graph, and relationships in the opposite direction.

This is a general purpose template engine that maps on top of a graph structured dataset. In this case, we assume that the CTI database is stored on a neo4j graph database, but this is not mandatory.

For this project we develop a triple generation architecture centered on a template language, template parser, and triple generation from results returned by the CTI DB. This architecture allows the user to write any template that connects a subgraph with *explicit paths* given (i.e. following explicit routes, as in *node.rel.rel.rel.property*) or *implicit paths* that are resolved using graph search by the underlying structured DB.

1.1. Overall Approach

The high-level approach or IFT triple generation for a template follows this flow:

1. The user formulates a template and submits it for generation. The template includes content-query+format blocks enclosed in { and }.
2. The template parser compiles all {.} blocks into a parse tree.
3. The query generator evaluates the parse tree and creates a Cypher query for the CTI graph DB.
4. The graph DB executes the query and returns a result set.
5. The triple generator evaluates again the parse tree and extracts data from the result set to generate one or more triples.
6. All generated triples, the original template, and associated metadata are saved to a new JSON file.

1.2. Document Structure

We continue with the Requirements specification for the template language and IFT triple generation system, including sample templates. That is followed by a review of the schema of the Cat. 1 CTI documents that informs template writers of concepts and relationships. The Design Section provides details of the overall system architecture, including the template language, parsing mechanisms, and general approach for triple generation.

2. Requirements

2.1. CTI Source Documents

A. The IFT Dataset Generation System should support the CTI document categories specified in the Athena-CTI-LLM Statement of Work, as follows:

   1. Category 1: structural/graph generation: MITRE ATT&CK, CVE, CAPEC, CWE, MITRE Engage, EPSS, CISA KEV
   2. Category 2: LLM-assisted generation: SIGMA Rules, MITRE CAR, Wazuh rules
   3. Category 3: proprietary datasets: Flashpoint Ignite Data

2.2. Template Language Requirements

A. The template format is determined by a language that imposes a well defined structure required by the triple generator algorithms.

B. The template language has an associated version string since it will evolve over time. Backward compatibility is not required.

C. The system should generate IFT triples in the form (*instruction*, *question*, *answer*), where:

   1. *instruction* includes instructions to the model on how it should answer the current question. The *instruction* part is passed as a chat template element (e.g. for the *assistant* or *system* roles) to the LLM generation service or call. A good *instruction* section could read like: "*You are a cybersecurity expert that has been trained to give precise responses to complex cybersecurity questions. You work in a SOC protecting data for enterprise customers helping to protect their digital assets.*"
   2. *question* is a text that represents a question that would be passed to the CTI LLM when deployed.
   3. *answer* is the answer expected from the model when asked the *question*, following the given instructions.

D. A template is a sequence of fields delimited by the labels "Instruction:", "Question:", and "Answer:". A field can be a:

   1. a text literal: text reproduced identically into the generated string
   2. a query field delimited by { and } that specifies a node, one of its properties or a constraint on those. A query field referring to a node (directly or indirectly, i.e. following relationships)

branches the template on a new dimension, leading to a new combination of generated strings.

3. a list comprehension delimited by [and] that allows generation of lists of strings merged in the same resulting text. A list comprehension includes one or more *text fields* and *query fields*.

E. **Template branching:** A template that contains *query fields* $\{a_i\}$, for i=1..n, and a_i are different node types in the CTI schema, should generate a triple ***for each possible combination*** of nodes $m_1, m_2, \ldots, m_n$ (where m_i is a node of type a_i) that are connected by (indirect or direct) relationships (edges) or any length. In other words, a triple is generated for any combination of $m_1, m_2, \ldots, m_n$ that belong to a connected subgraph of the CTI concept graph.

1. Example: a simple template "{attack-pattern}" generates a new triple text for each *attack-pattern* node. The template *branches* on the *attack-pattern* node type.
2. Example: a template "{attack-pattern} {tool}" generates a new triple string for each possible combination of (*attack-pattern, tool)* pairs that are connected by the relationship *tool - uses -> attack-pattern*. It generates triples for all *attack-pattern*s, but also for all possible *tool*s, like a Cartesian product, constrained by the relationship.
3. We can say that the addition to a template of a field with a new node type adds a new branch on a new dimension to the generation process.
4. Multiple query fields referring to the **same node type** resolve **to the same node of that type** and the template will not branch on that node type.
    a. Example: "{attack-pattern.name} {attack-pattern.id}" results in a sequence of triples with all possible attack-patterns "attack-pattern.name attack-pattern.id". The template branches only once on the attack-pattern.

F. ***A query field*** in a template refers to CTI concepts from the documents listed above in the following way.

1. Each query field must refer to exactly one node and maximum one node property {*a.p*} or relationship {*a.relation*}. If a query field specifies just a node, as in {attack-pattern} then the default property generated is a configurable parameter, by default, its name.
2. It may use standardized aliases that map meaningful names to CTI concept types from the ASG CTI DB schema. For example: a *technique* is an *attack-pattern* and a *mitigation* is a *course-of-action*. "*technique"* and "*attack-pattern"* can be used interchangeably.
3. It may refer to attributes at deeper levels from the root node. For example, an ATT&CK *attack-pattern* node (i.e. *technique*) embeds attribute *x_mitre_detection* (string) and attribute *kill_chain_phases*, a list of dictionaries, in the JSON source document:

    [{'kill_chain_name': 'mitre-attack', 'phase_name': 'defense-evasion'},

    {'kill_chain_name': 'mitre-attack', 'phase_name': 'privilege-escalation'}]

    Hence, one can refer in a template to *technique.x_mitre_detection* and *technique.kill_chain_phases*[0].*phase_name*.
4. It may use filters for concept property values. For example this can be useful to query the CTI DB for tools on a particular platform or for sub-techniques.

a. A filter can use these operators: = (equal), != (not equal).

b. An “any” filter [? op value] describes a condition that is necessary to apply to at least one values in a list property. E.g. *attack-pattern.x_mitre_platforms*[? = ”Linux”].

c. An “all” filter [* op value] describes a condition that is necessary to apply to all values in a list property. E.g. attack-pattern.x_mitre_platforms[* != “Windows”]

5. Multiple query fields may refer to more than one node type. In this case, these nodes are resolved to concepts connected in the concept graph using additional path or filter constraints in the current template or using an implicit path between the concepts.

   a. A template that contains {a.prop_a} and {b.prop_b} where *a* and *b* are ***node types*** resolves to strings (e.g. triples) with all possible combinations of *node_a.prop_a* and *node_b.prop_b*, where *node_a* is from type *a* and *node_b* is from type *b*, and *node_a* is connected with a path to *node_b* or vice-versa [1]. The addition of any new node type to a template “branches” or “splits” the template on an extra dimension. This is similar to a Cartesian product filtered by a relation between the current two nodes.

6. A query field may define a **local-scope variable** that can be used elsewhere in the same template to refer to the same node. A variable repeated in other query fields carries its identity, referring to the same node.

   a. An **unbound variable** becomes an alias for a node if its type and it becomes a branching node. Example: in “{m:malware.name} {m.id}” variable *m* is unbound in the first field and takes values across all possible *malware* nodes (not their names). In the second field, *m* refers to the same malware node as in the first field and it is **bound** to it.

   b. A variable can be bound to a node resulting from a relationship path. Example: in {***intset***:campaign.attributed-to.name} *intset* is a variable bound to one of the *intrusion-set*s associated with the *campaign*.

   c. Two different unbound variables of the **same type** resolve to **different** concepts/nodes in the graph DB. Example: “{t1:tool} is not {t2:tool}”, *t*1 and *t*2 are different *tool* nodes, causing branching first on *t*1 and then on *t*2,

   d. Two **unbound** variables of **different** types are resolved to concepts connected in the concept graph using additional path or filter constraints in the current template or using an implicit path between the concepts, as described in statement F.5.a, above.

7. A query field may refer to list properties and select a specific entry by index, *node.property*[0], or may use a wildcard “?” to specify a random pick, *node.property*[?]. Referring to a single value of a property does not lead to branching a template.

8. They can use inequality operators between bound node variables to indicate that two nodes are different. This is required by negative examples. This type of filter constrains template branching.

---

[1] Formally, a template $\tau = (a.p, b.q)$ with properties $p \in \Pi_a$ (the set of *a*’s properties) and $q \in \Pi_b$ leads to a generation of sequences $\gamma(\tau) = \{(n_a.p\, n_b.q) \mid n_a \in a \wedge n_b \in b \wedge R(n_a, n_b)\}$ where predicate R(x, y) is true iff there is a route between nodes x and y.

a. Two different techniques used by the same tool: {t1:tool.uses} {t2:tool.uses} {t1 != t2}. This is the default behavior.

b. A technique not used by a specific tool: {t1:tool.uses} {t2:attack-pattern != t1}

G. A template may use invisible sections marked properly that may define and bind variables and set constraints used elsewhere in the template. An invisible section of a template is delimited by **<*** and ***>** and the text inside is excluded from generation. Invisible sections are used to bind variables and filter on relationships of conditions involving concept properties, used elsewhere in the same template.

H. A template may include a **list comprehension**, a sequence of query fields with cardinality greater than 1 that results in a list of concatenated strings included in the same generated triple. It is written between **[** and **]**. It effectively collects template branches into the same generated string instead of causing separate triples for each branch. This is usable, for instance, to generate text about a *tool* that *uses* a list of *techniques,* or a *malware* that uses a *technique* mitigated by multiple *course-of-action*s.

1. Assuming the *tool* node type branches into three *tool* nodes, notation [abc {tool} xyz,] generates string “abc tool1 xyz, abc tool2 xyz, abc tool3 xyz”, a single triple.

2. Notation [{*tool.uses*[*].*name},*] results in a comma-separated list with the names of all techniques (attack-patterns) used by *tool*. List comprehensions don’t lead to template branching.

3. Example [{m:malware.name} uses technique {m.uses.name}\n] branches first on *m:malware* and then on *m*’s techniques, generating a number of lines with all possible combinations of malware and the malware’s used techniques.

4. Nested list comprehensions are not supported.

I. They may refer to a directed relationship between two concepts and also to the inverse relationship. For example a *course-of-action* mitigates an *attack-pattern* and, also, an *attack-pattern* is mitigated-by a *course-of-action*. *uses → used_by, mitigates → mitigated_by, revoked-by → revokes, detects → detected_by*, etc.

J. The template specification language should allow precise identification of CTI concepts that are common in multiple CTI source models, such as *mitigation*, that exists in CAPEC and also in ATT&CK. A notation for disambiguation uses prefix ***source#*** notation. E.g. ***attck#****mitigation* vs. ***capec#****mitigation*. Source prefixes may be specified in the template generation job configuration file.

2.3. Template Examples

A. The main objective of these examples is to illustrate the capabilities of the template language and to elicit comments and suggestions from the reader. As such, templates are very short and they may not be representative of actual templates CTI domain experts will write for IFT. The *instruction* section is particularly short to not take additional space.

B. It is important to understand that the purpose of the template generation approach is to allow a wide range of templates to be supported.

C. The following templates use the standard concept, attribute, and relationship names as defined in the source documents. It does not use the current terminology from the CTI DB schema since that is likely to evolve. In contrast, production templates will refer exclusively to terms defined by the CTI DB schema.

1. Simple example with node attributes:

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> *Question*:
> Describe the technique {attack-pattern.name} ({attack-pattern.id}) and give a method for its detection.
>
> *Answer*:
> This is the description of {attack-pattern.name}:
> {attack-pattern .description}
> Its method of detection is {attack-pattern.x_mitre_detection}

2. Template example that links description to attack pattern and id:

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> *Question*:
> This attack pattern description is representative of which attack pattern name and id?
> {attack-pattern.description}
>
> *Answer*:
> The given description belongs to {attack-pattern.name} with id {attack-pattern.id}.

3. Template example that filters technique nodes using a property value for platform and also filter techniques that are sub-techniques. It uses variables (e.g. *m* in m:mitigation) bound here to a mitigation (course-of-action) node. Variable *t* is bound to *m*'s associated attack-pattern (or technique) node. This demonstrates constraining variables to follow a relationship, in this case, *course-of-action – mitigates -> attack-pattern*. A second constraint forces the technique's platform to be "Windows" using the "any is equal to" filter t.x_mitre_platforms[? = 'Windows']:

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
Which subtechniques on Windows are mitigated by {m:mitigation.name}?

Answer:
Mitigation {m.name} ({m.id}) mitigates technique {t:m.mitigates.name} that has platform {t.x_mitre_platforms[? = 'Windows']}.

4. Example with relationship that refers to a random element of a list property using [?] any wildcard. Expression t.x_mitre_platforms[?] results in just one platform attribute string being generated, i.e. no branching/split

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
Give the name, ID, description, and detection method of an attack pattern that uses tool {t:tool.name} on platform {t.x_mitre_platforms[?]}.

Answer:
Tool {t.name} ({t.id}) on platform {t.x_mitre_platforms[?]} uses attack pattern {ap:t.uses.name} with ID {ap.id}.
Its detection method is {ap.x_mitre_detection}

{ap.description}

5. A template that navigates two relationships and generates one combination out of several possible:

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
List a data component that is needed by mitigation {m:mitigation.name} according to MITRE ATT&CK.

Answer:
Since mitigation {m.name} ({m.id}) mitigates technique {t:m.mitigates.name} ({t.id}) and {t.name} is detected by {dc:t.detected-by.name} ({dc.id}), the correct answer is: {dc.name} ({dc.id}).

6. A template example that generates a list of related concepts using a notation similar to a list comprehension. Inside the list comprehension **[...]** variable *t* is bound to all techniques:

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> *Question*:
> List the names of all data components that are involved by mitigation {m:mitigation.name}.
>
> *Answer*:
> **[**Mitigation {m.name} ({m.id}) mitigates technique {t:m.mitigates[*].name} ({t.id}) and {t.name} is detected by {t.detected_by.name}.
> **]**
> Hence, the correct answer is:
> [{m.mitigates[*].detected_by.name},]

7. Template that refers to relations involving two different concepts of the same type constrained by platform type. The two techniques below are different nodes linked to the same malware. Note how variables *t1* and *t2* are bound in the *answer* section, after they are first used in the

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> *Question*:
> Based on ATT&CK, what are the malwares that use both techniques {t1.name} and {t2.name} on the Linux platform?
>
> *Answer*:
> Malware {m:malware.name} ({m.id}) uses technique {t1:m.uses.name} ({t1.id}) on {t1.x_mitre_platforms[? ='Linux']} and
> malware {m.name} uses technique {t2:m.uses.name} {t2.id} on {t2.x_mitre_platforms[? ='Linux']}.
> Therefore, the correct answer is {t1.name} and {t2.name}.

8. Example with source disambiguation (capec#), needed here since *attack-pattern* is both a ATT&CK and a CAPEC concept.

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
The CAPEC attack pattern {cap:capec#attack-pattern.name} affecting the memory resource indicates that:
{cap.Description}.
What are some detection methods to prevent it?

Answer:
The CAPEC attack pattern {cap.name} ({cap.id}) has a related weakness {cwe.name} ({cwe.id}) which affects the {cwe:cap.related-weakness.Affected_Resources[? = "Memory"]} resource. This weakness has the following detection methods:
[{dm:cwe.Detection_Method[*].Method}, with id {dm.id}, effectiveness {dm.Effectiveness}:
{dm.Description}
]

9. Example of a template that uses an *invisible* section to set up bindings and constraints for relationships across CTI source documents. It is delimited by <* and *> to impose the sub-technique constraint. No text will be generated for anything enclosed in an invisible section.

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
Consider this CVE vulnerability:
{cve.descriptions[0].value}
Identify the common vulnerability name and id.

Describe the corresponding CWE weakness name and suggest a CAPEC attack pattern with a **high likelihood of attack**. Give the CAPEC name and id and also name of the ATT&CK technique implements that attack pattern.

Answer:
<* this invisible section binds variables to concepts and forces some constraints:
{capec:capec#attack-pattern.Likelihood_Of_Attack="**High**"}
{ap:attck#attack-pattern.external_references[?].source_name="mitre-attack"}
*>
The CVE name is {cve:cwe.Observed_Example[?].title} with id {cve.cveId}.
A related CWE weakness for this vulnerability is {cwe:capec.Related_Weakness[?].Name} with id {cwe.ID}.
A CAPEC attack pattern using this weakness is {capec.Name} with id {capec.ID}.
An ATT&CK technique that implements this attack pattern is {ap.name|} ({ap.id}).

10. The example from Cyberpal.AI article, Figure 2. Note that ATT&CK Enterprise has no relationships linking tactics to techniques.

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> *Question*:
> Which tactic is related to the following application of malware {m:malware.name}:
> {m.usage}?
>
> *Answer*:
> {m.description}
>
> {m.name} is identified by MITRE ID {m.id}.
>
> The provided usage of {m.name} ({m.id}) pertains to the MITRE sub-technique {t:technique.sub.name} ({t.sub.id}). This is since the given sub-technique {st:t.sub-technique.description}.
>
> Sub-technique {st.name} ({st.id}) is a type of tactic {tactic.name}({tactic.id}).
> Therefore, the answer is: tactic {tactic.name} ({tactic.id}).

11. A negative example template. The constraint is placed in an invisible section.

> *Instruction*: You are a CTI expert that gives precise and concise answers.
>
> <* {m:malware} {t:m.uses} {dc:x-mitre-data-component} {t.detected_by[* != dc]} *>
>
> *Question*:
> Does malware {m.name} use a technique that is detected using {dc.name}?
>
> *Answer*:
> No. {m.name} uses these techniques:
> [{t:m.uses.name},]
> None of these techniques are detected by {dc.name}.

12. A negative example.

Instruction: You are a CTI expert that gives precise and concise answers.

Question:
What are some mitigations that do not address technique {t:attack-pattern.name}?

Answer:
Here are some mitigations that do not mitigate technique {t.name}:
[{m.name}, ({m.id}) <* {m:mitigation} {t.mitigated_by[* != m]} *>
]

2.4. Incremental Generation; CTI Document and Concept Versions

A. The CTI source documents are characterized by these relevant properties:
   1. current version number
   2. creation and last modified timestamp.

B. In general, each concept in these documents has creation and last modified timestamps. These properties must be preserved when imported to the ASG CTI DB and they should be usable when specifying queries.

C. The IFT triple generator tools and API should support incremental generation in the following way:
   1. The user can specify the version number interval [min-version, max-version] to be used for each source document. Both ends of the interval are optional. Min. end defaults to the first version and max. end defaults to the current (most recent) version of the current document.
   2. The user can specify the last modified [min-date, max-date] interval to be used for each source document. Both ends of the interval are optional. Min. end defaults to creation date and max. end defaults to the current date or some reference date in the future.
   3. The generation tool processes each given templates by filtering concepts based on version interval and last modified interval as follows:
      a. any relationship a - r -> b between concepts a and b is used for generation only if
         1. (a.document.version is in [min-version, max-version] or b.document.version is in [min-version, max-version]) and
         2. (a.document.modified-date is in [min-date, max-date] or b.document.modified-date is in [min-date, max-date]) and
         3. (a.modified-date is in [min-date, max-date] or b.modified-date is in [min-date, max-date])

2.5. Input Generation Configuration File

A. The user creates a JSON file that is submitted to the triple generation tool or API. This file must include these information:
   1. Version number interval of the CTI source documents used for generation.
   2. Last modified date interval of the CTI source documents used for generation.
   3. Dataset version string.
   4. Generated dataset name string. E.g. “ATT&CK, CVE, and CWE”.
   5. Output directory path name where the generated triples will be saved.
   6. Template language version string.
   7. A template descriptor object (dictionary) for each template with these items:
      a. template name: a human readable name for this template that will be used to more easily identify the generated triples. It must be devoid of special characters and spaces since it will be used for naming files with generated triples. This is optional and, if missing, it will be replaced by a randomly generated string. [As an option, the default could be created using the dataset name stripped of special characters.]
      b. template text
      c. count limit: maximum number of triples to be generated for the current template
      d. fraction coverage limit. A template could be mapped to a potentially large number of possible matching triples. The fraction coverage limit stops generation from this template after reaching a triple count that is equal to the given fraction of that maximum.
         1. When both the count limit and the fraction coverage limit are given, generation stops after the lower of those two.

2.6. The Generated Dataset Format and the Report File

A. The dataset generation code places all files in the output directory path specified in the job configuration file.
   1. [TODO: should it erase all prior files from that directory before generation?]
B. The initial job configuration JSON file is copied to the output directory.
C. The generation code creates a new report file in JSON format that describes the following:
   1. The path name of the generation job configuration.
   2. Generation run type: generation run or just a test run (argument *test_only* False/True).
   3. Generation results as a dictionary:
      a. “error_count”: number of template parsing and generation errors
      b. “warning_count”: number of templates with parsing or generation warnings. A *warning* means that triples were generated from a template with some non-fatal errors.

c. “clean_count”: number of templates used for generation without errors or warnings

4. Number of templates.
5. Total total number generated triples.
6. Version number for the CTI source documents used for generation.
7. Last modified date of the CTI source documents used for generation.
8. Version number interval of the CTI source documents from the job configuration file.
9. Last modified date interval of the CTI source documents from the job configuration file.
10. Generated dataset version string from the job configuration file.
11. Template language version string from the job configuration file.
12. Generated dataset name string. E.g. “ATT&CK, CVE, and CWE”, from the job configuration file
13. Original output directory path name, from the job configuration file.
14. A JSON list with generation reports (each a JSON object) for each template, with these items:
    a. template text
    b. template name
    c. list with generation errors and warning (e.g. from parsing). [TODO: format]
    d. count limit, from 2.A.7.c
    e. fraction coverage limit, from 2.A.7.d
    f. number of generated triples
    g. coverage fraction of generated triples. E.g. 1.0 if all possible triples were generated, 0.0 if none were generated.
    h. path with file name with the generated triples corresponding to the current template relative to this report file’s path. The fine-tuning code use these paths to assemble a batch of triple text before the actual model fine-tuning process.

D. All triples generated from a particular template are saved to a separate JSON file stored in the output directory. The file name is derived from the template name, if provided, or from the generation job name. Notice some information is repeated from the report file. The generated file contains:

1. Template name.
2. Current template text.
3. Version number for the CTI source documents used for generation.
4. Last modified date of the CTI source documents used for generation.
5. Version number interval of the CTI source documents from the job configuration file.

6. Last modified date interval of the CTI source documents from the job configuration file.
7. Generated dataset version string from the job configuration file.
8. Template language version string from the job configuration file.
9. Number of generated triples for the current template.
10. Coverage fraction of generated triples for the current template.
11. A list of triples with this dictionary format:
    a. "instruction": the instruction text
    b. "question": the generated question text
    c. "answer": the answer text

2.7. IFT Data Generation API and Tools

A. The generation API has a function ***generate_triples(job_file_name:str, test_only:bool)*** that takes as argument a generation job descriptor file name and a Boolean *test_only*.
    1. If parameter *test_only* is *False* then the function generates the triples based on the information from the generation job description given in file *job_file_name.* A generation report file and files for each template will be created according to the requirements in preceding sections.
    2. *If test_only* is *True* then the function will only parse the templates, query the CTI DB, count triples that would be generated without actually generating the triples' text and individual template-specific files. A generation report file is created as described above, with the generation result reflecting this was a test only.
    3. This function returns the dictionary (JSON object) with the generation report.

B. A standalone Python script ***generate_triples.py*** takes as command line arguments the job file name and the test_only flag and calls the ***generate_triples*** function.

C. [TODO: add support for parallel execution for generation algorithms. The size of a generated dataset is unknown at this time, as is the CPU time needed to generate triples. The main limitation would be the ability of the currently used neo4j graph DB to run queries in parallel in multi-threaded code or multi-process code. The size and scalability of the production-level DB are also unknown at this time.]

D. The generation API has a function ***format_report(gen_report_file_name:str, format_type:str)*** that takes as argument a generation report JSON file and creates and returns a string in a human-readable format, such as text tables, HTML, or XML, as specified by the second argument:
    1. Parameter *gen_report_file_name* is the generation report JSON file name.
    2. Parameter *format_type* is a string that could be "text" for a textual report representation, "html" for an HTML format, etc. A trivial "json" format type could simply return the report content as a string.

3. Data Schema

NOTES:

- the schema diagrams below are incomplete.
- KVE and EPSS schemas are not included yet.

3.1. MITRE ATT&CK SCHEMA

The ATT&CK schema has a graph structure where key CTI concepts are nodes with attributes and relationships between these concepts form directed graph edges.

The up to date ATT&CK dataset (Enterprise, Mobile, and Industrial/Control versions) can be downloaded as JSON files from the MITRE STIX Github repository.

The schema below was extracted from the June 2025 Enterprise version of the ATT&CK JSON file.

NOTE: the ATT&CK Enterprise JSON file does not include any explicit relationships that refer to **x-mitre-tactic** objects. However relationships linking techniques to one tactic are defined by the "kill_chain_phases/[]/phase_name" property of an attack-pattern, taking the value of a tactic's property x_mitre_shortname. The current JSON graph parser does not handle that case yet and it will be fixed in a future version.

A draft graph with the concepts and relationships is displayed next, in Figure 1. The x-mitre-tactic nodes are omitted from the diagram.

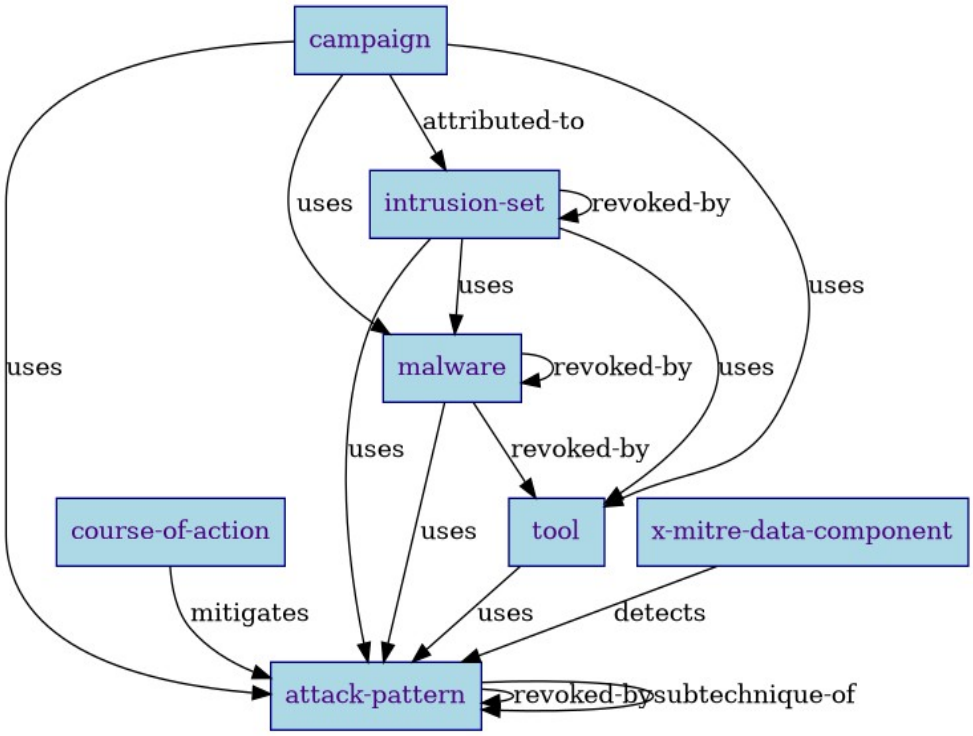

Figure 1: MITRE ATT&CK schema graph extracted from 06/2025 JSON source document.

The following table lists the concepts (nodes) with properties and the relationships extracted from the same 6/2025 JSON ATT&CK document. The rightmost column includes the count for nodes and relationships. Note that _count is not a real property.

Concept:

Node	Property	Comments
attack-pattern	_count	_count = 823
	created	
	created_by_ref	
	description	
	external_references	
	id	
	kill_chain_phases	
	modified	
	name	
	object_marking_refs	
	spec_version	
	type	
	x_mitre_attack_spec_version	
	x_mitre_data_sources	
	x_mitre_deprecated	
	x_mitre_detection	
	x_mitre_domains	
	x_mitre_is_subtechnique	
	x_mitre_modified_by_ref	

```
                    x_mitre_platforms
                    x_mitre_version
campaign            _count                                  _count = 47
                    aliases
                    created
                    created_by_ref
                    description
                    external_references
                    first_seen
                    id
                    last_seen
                    modified
                    name
                    object_marking_refs
                    revoked
                    spec_version
                    type
                    x_mitre_attack_spec_version
                    x_mitre_deprecated
                    x_mitre_domains
                    x_mitre_first_seen_citation
                    x_mitre_last_seen_citation
                    x_mitre_modified_by_ref
                    x_mitre_version
course-of-action    _count                                  _count = 268
                    created
                    created_by_ref
                    description
                    external_references
                    id
                    modified
                    name
                    object_marking_refs
                    spec_version
                    type
                    x_mitre_attack_spec_version
                    x_mitre_deprecated
                    x_mitre_domains
                    x_mitre_modified_by_ref
                    x_mitre_version
intrusion-set       _count                                  _count = 181
                    aliases
                    created
                    created_by_ref
                    description
                    external_references
                    id
                    modified
                    name
                    object_marking_refs
                    revoked
                    spec_version
                    type
                    x_mitre_attack_spec_version
                    x_mitre_contributors
                    x_mitre_deprecated
                    x_mitre_domains
                    x_mitre_modified_by_ref
                    x_mitre_version
malware             _count                                  _count = 667
                    created
```

```
                        created_by_ref
                        description
                        external_references
                        id
                        is_family
                        modified
                        name
                        object_marking_refs
                        revoked
                        spec_version
                        type
                        x_mitre_aliases
                        x_mitre_attack_spec_version
                        x_mitre_deprecated
                        x_mitre_domains
                        x_mitre_modified_by_ref
                        x_mitre_platforms
                        x_mitre_version
tool                    _count                                  _count = 91
                        created
                        created_by_ref
                        description
                        external_references
                        id
                        modified
                        name
                        object_marking_refs
                        revoked
                        spec_version
                        type
                        x_mitre_aliases
                        x_mitre_attack_spec_version
                        x_mitre_contributors
                        x_mitre_deprecated
                        x_mitre_domains
                        x_mitre_modified_by_ref
                        x_mitre_platforms
                        x_mitre_version
x-mitre-data-component  _count                                  _count = 109
                        created
                        created_by_ref
                        description
                        id
                        modified
                        name
                        object_marking_refs
                        revoked
                        spec_version
                        type
                        x_mitre_attack_spec_version
                        x_mitre_data_source_ref
                        x_mitre_deprecated
                        x_mitre_domains
                        x_mitre_modified_by_ref
                        x_mitre_version

x-mitre-tactic          _count                                  _count = 14
                        created
                        created_by_ref
                        description
                        external_references
```

	id	
	modified	
	name	
	object_marking_refs	
	spec_version	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_domains	
	x_mitre_modified_by_ref	
	x_mitre_shortname	
	x_mitre_version	

Edges:

Edge Type	Property	Comment
attack-pattern revoked-by attack-pattern	_count	132
	created	
	created_by_ref	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
attack-pattern subtechnique-of attack-pattern	_count	470
	created	
	created_by_ref	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
campaign attributed-to intrusion-set	_count	23
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	

campaign uses attack-pattern	_count	902
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
campaign uses malware	_count	82
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
campaign uses tool	_count	58
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
course-of-action mitigates attack-pattern	_count	1421
	created	
	created_by_ref	
	description	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	

	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
intrusion-set revoked-by intrusion-set	_count	6
	created	
	created_by_ref	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
intrusion-set uses attack-pattern	_count	4046
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
intrusion-set uses malware	_count	614
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
intrusion-set uses tool	_count	438
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	

	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
malware revoked-by malware	_count	1
	created	
	created_by_ref	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
malware revoked-by tool	_count	1
	created	
	created_by_ref	
	description	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	revoked	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
malware uses attack-pattern	_count	9303
	created	
	created_by_ref	
	description	
	external_references	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
tool uses attack-pattern	_count	798
	created	
	created_by_ref	
	description	
	external_references	
	id	

	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	
x-mitre-data-component detects attack-pattern	_count	2116
	created	
	created_by_ref	
	description	
	id	
	modified	
	object_marking_refs	
	relationship_type	
	source_ref	
	spec_version	
	target_ref	
	type	
	x_mitre_attack_spec_version	
	x_mitre_deprecated	
	x_mitre_modified_by_ref	

Total: 7 nodes, 16 edges

3.2. MITRE CWE SCHEMA

Common Weakness Enumeration records describe common software and hardware weakness types that could have security ramifications. A weakness is a condition in a software, firmware, hardware, or service component that, under certain circumstances, could contribute to the introduction of vulnerabilities. Weakness conditions are in many cases introduced by the developer during development of the product.

As of 9/2025 the current version of the CWE dataset is 4.18. The schema version is 7.2.
A CWE links to CVE records, to CAPEC records, and to mitigations.

The CWE schema is described on this webpage. The corresponding XML Schema Document (XSD) file for this version is posted here.
CWE records are stored in XML files on this website.

CWE XML files are organized in Views. A View groups together in a single .xml file records that are similar from some perspective, such as “software development” (699.xml), or “Software Fault Pattern (SFP) Clusters” (888.xml). It is possible that one CWE record belongs to only one XML file. Views are further described in the schema document.

Of interest to us may be the Taxonomy_Mappings enumeration. From the schema doc:
The TaxonomyMappingsType complex type is used to provide a mapping from an entry (Weakness or Category) in CWE to an equivalent entry in a different taxonomy. The required Taxonomy_Name

attribute identifies the taxonomy to which the mapping is being made. The Entry_ID and Entry_Name elements identify the ID and name of the entry which is being mapped. The Mapping_Fit element identifies how close the CWE is to the entry in the taxonomy.
It maps CWEs to Object Management Group's OMG ASCMM, OMG ASCRM, OMG ASCPEM, or some category like ""7 Pernicious Kingdoms", maybe from CWE.

Key concepts and relationships:

Below is a list of the relevant XML paths. Paths with names starting with @ indicate XML element attributes. A number after / indicates an enumeration (or sequence) of elements.
Relationships are highlighted in yellow.

Weakness_Catalog/@Date	release date, yyyy-mm-dd, of this document
Weakness_Catalog/@Name	string, document name, may be a VIEW name
Weakness_Catalog/@Version	string, version "xx.yy"

A weakness is described in this enumeration:
Weakness_Catalog/Weaknesses/Weakness/0

.../@ID	int, CWE record ID
.../@Name	string, weakness name
.../@Diagram	string, path or URL to diagram image
(@Structure and @Status omitted)	
.../Affected_Resources/Affected_Resource	string, e.g. "Memory"
.../Alternate_Terms/Alternate_Term/0	aliases
.../Description	string
.../Term	string, the alias, e.g. "Tapjacking"
.../Applicable_Platforms/Language/0	
.../@Class	string, language class, e.g. "Not Language-Specific"
.../@Prevalence	string, e.g. "Rarely"
.../@Name	string, programming language name, e.g. "C++"
.../Common_Consequences/Consequence/0	
.../Impact	string, e.g. "Varies by Context", "Read Memory"
.../Scope	string, e.g. "Confidentiality", may repeat
.../Note	string, useful contexts
.../Content_History/Modification/0	
.../Modification_Name	
.../Modification_Date	date, yyyy-mm-dd
.../Modification_Organization	
.../Modification_Comment	
.../Modification_ReleaseDate	date, yyyy-mm-dd

.../Content_History/Submission	
.../Submission_Date	date, yyyy-mm-dd
.../Submission_Name	e.g. “CWE Content Team”
.../Submission_Organization	e.g. “MITRE”
.../Submission_ReleaseDate	
.../Submission_Version	
.../Demonstrative_Examples/Demonstrative_Example/0	
.../Intro_Text	
.../Body_Text	
.../Example_Code	
.../Example_Code/@Nature	string, e.g. “Bad”
.../Example_Code/@Language	string, e.g. “C++”, “C”
.../Description	string, short description text
.../Detection_Methods/Detection_Method/0	enumeration
.../@Detection_Method_ID	string, optional
.../Description	string
.../Effectiveness	string, e.g. “Soar Partial”
.../Method	string, method title
.../Extended_Description	string, longer description text, multiple paragraphs
.../Functional_Areas/Functional_Area	string, e.g. “Memory Management”
.../Likelihood_Of_Exploit	
.../Modes_Of_Introduction/Introduction/0	enumeration
.../Phase	string, e.g. “Architecture and Design”, Implementation
.../Note	string
.../Observed_Examples/Observed_Example/0	**enumeration, points to related CVE**
.../Reference	string, CVE_ID
.../Link	URI, link to CVE record
.../Description	string
.../Potential_Mitigations/Mitigation/0	**enumeration, node**
.../@Mitigation_ID	string, e.g. "MIT-36"
.../Description	string, may include <xhtml:p> paragraphs
.../Effectiveness	string
.../Effectiveness_Notes	string, may include <xhtml:p> paragraphs
.../Phase	string, may repeat, e.g. “Implementation”
.../Strategy	string, e.g. “Input Validation”
.../References/Reference/0	enumeration
.../@External_Reference_ID	string, format “REF-nnn”

.../@Section	string, e.g. “Chapter 20, "”
.../Related_Attack_Patterns/Related_Attack_Pattern/0	enumeration
.../@CAPEC_ID	int, format “nnnn”
.../Related_Weaknesses/Related_Weakness/0	enumeration with related CWEs
.../@CWE_ID	int, CWE ID of related weaknesses
.../@Nature	string, rel. type, “ChildOf”, “CanPrecede”, “PeerOf”
.../@Ordinal	string, “Primary”
.../@View_ID	integers
.../Taxonomy_Mappings/Taxonomy_Mapping/0	numeration,
.../@Taxonomy_Name	string, e.g. “Software Fault Patterns",
.../Entry_ID	string, taxonomy specific ID
.../Weakness_Ordinalities/Weakness_Ordinality	
Weakness_Catalog/Categories/Category/0	**enumeration with weakness categories**
.../@ID	int, category ID
.../@Name	string
.../@Status	string, e.g. “Draft”
.../Relationships/Has_Member/0	enumeration with related CWEs in this category
.../@CWE_ID	int, related CWE ID
.../@View_ID	int, View ID
.../Content_History/Modification/0	enumeration with modification records
Weakness_Catalog/External_References/External_Reference/0	enumeration
.../@Reference_ID	string, reference ID, e.g. “REF-12c87”
.../Author/0	string, author names
Weakness_Catalog/Views/View	omitted

3.3. MITRE CVE SCHEMA

The CVE record catalog lists standardized identifiers, called CVE IDs, for specific, publicly known cybersecurity vulnerabilities and exposures in software, hardware, firmware, or services. Each record provides a brief description of the weakness, its type, and references to information needed to understand and address the flaw.
The Common Vulnerabilities and Exposures (CVE) webpage links to the CVE schema repository on Github.
Information is stored in CVE Records in JSON format. The format is described at https://cveproject.github.io/cve-schema/schema/docs/ .

CVE main download page.
CVE JSON Github repository.

CVE Record structure in a nice Mindmap diagram.
An Avanced Example JSON entry.

Information in a CVE Record has a tree structure represented by JSON dictionaries + lists.

Relevant properties / concepts:
id, CNA, affected things, references, dates, credits, CVSS scores/metrics, vulnerability types, solutions, impacts, exploits.

Key concepts and relationships:

Relationships are highlighted in yellow.

cveMetadata/cveId: string, assigned by CNA
cveMetadata/datePublished: datetime
cveMetadata/dateUpdated: datetime

containers/cna/title: string, title, headline, or a brief phrase summarizing the CVE record

containers/cna/affected: array, with affected products
containers/cna/affected/[]/product: string, product name
containers/cna/affected/[]/vendor: string
containers/cna/affected/[]/defaultStatus: affected or unaffected
containers/cna/affected/[]/versions: array, with version intervals for affected/unaffected

containers/cna/descriptions: array
containers/cna/descriptions/[]/lang: string, e.g. “en”, multilingual support.
containers/cna/descriptions/[]/value: string – English description
containers/cna/descriptions/[]/supportingMedia: array – multimedia descriptions, e.g. HTML, or images

Relationship:
containers/cna/impacts: array, consequences of a successfully exploited vulnerability, links to CAPEC records
containers/cna/impacts/[]/capecId: string, ID of CAPEC record with attack pattern
containers/cna/impacts/[]/descriptions: array, with CAPEC ID and title

containers/cna/metrics[0]: CVSS metrics, key=CVSS version (e.g. 4.0)

containers/cna/problemTypes: array, links to CWE records.
containers/cna/problemTypes/[]/descriptions: array
containers/cna/problemTypes/[]/descriptions/[]/cweId: string, CWE record ID
containers/cna/problemTypes/[]/descriptions/[]/description: string, CWE ID + title (?)
containers/cna/problemTypes/[]/descriptions/[]/lang: string, e.g. “en”

containers/cna/problemTypes/[]/descriptions/[]/type: string, e.g. “CWE”

containers/cna/configurations: array, Configurations required for exploiting this vulnerability.
containers/cna/configurations/[]/description, string

containers/cna/workarounds: array, Workarounds and mitigations for this vulnerability.
containers/cna/workarounds/[]/description: string

containers/cna/solutions: array, Information about solutions or remediations available for this vulnerability.
containers/cna/solutions/[]/description: string

containers/cna/exploits: array, Information about exploits of the vulnerability.
containers/cna/exploits/[]/description: string

containers/cna/references: array, reference data (URLs or embedded)

containers/cna/timeline: array, timeline information for significant events about this vulnerability or changes to the CVE Record.

containers/adp – information from Authorized Data Publishers , similar to containers/cna

3.4. MITRE CAPEC

The Common Attack Pattern Enumeration and Classification (CAPEC) effort provides a publicly available catalog of common attack patterns that helps users understand how adversaries exploit weaknesses in applications and other cyber-enabled capabilities.

An attack pattern is an abstraction mechanism for helping describe how an attack is executed. Each pattern defines a challenge that an attacker may face, provides a description of the common technique(s) used to meet the challenge, and presents recommended methods for mitigating an actual attack.

A CAPEC record provides a detailed description of an attack pattern that an adversary can use to exploit weaknesses in software and hardware.

Each record details the method and techniques behind a specific type of attack, often linked to a corresponding weakness in the Common Weakness Enumeration (CWE). Each attack pattern captures knowledge about how specific parts of an attack are designed and executed, and gives guidance on ways to mitigate the attack's effectiveness.
As September 2025 there are 559 attack patterns.

Comparison between CAPEC and ATT&CK.

CAPEC source documents are written in the XML format. The schema is described in English on this page, last modified in 01/2023.
The CAPEC XML Schema Document (XSD) formally defines the CAPEC document language. It is at version 3.5, as of 09/2025.

Key concepts and relationships:

Below is a list of the relevant XML paths. Paths with names starting with @ indicate XML element attribute names. A number between / and / indicates an enumeration (or sequence) of elements. **Relationships** are highlighted in yellow.

Attack_Pattern_Catalog/@Date	string, “yyyy-mm-dd”, date of this version
Attack_Pattern_Catalog/@Name	string, catalog name
Attack_Pattern_Catalog/@Version	string, e.g. “3.5”

Attack_Pattern_Catalog/Attack_Patterns/Attack_Pattern/0

...@Abstraction	string, “Detailed” or “Standard” or “Meta”
...@ID	int, CAPEC ID
...@Name	string, attack pattern name
...@Status	string, “Stable” or “Draft”

.../Consequences/Consequence/0	
.../Impact	string, e.g. Execute Unauthorized Commands
.../Scope	string, e.g. Availability, Confidentiality, Integrity
.../Note	string

.../Content_History/Submission	
.../Submission_Date	string, “yyyy-mm-dd”
.../Submission_Name	
.../Submission_Organization	

.../Content_History/Modification/0	
.../Modification_Name	string, who did it
.../Modification_Organization	
.../Modification_Date	string, “yyyy-mm-dd”
.../Modification_Comment	

.../Description	string
.../Example_Instances/Example/0	string, example
.../Execution_Flow/Attack_Step/0	
.../Description	string, step descriptions
.../Phase	string, e.g. “Explore”, “Experiment”
.../Step	int, step number, from 1

.../Likelihood_Of_Attack string, e.g. “Low”, “High”

.../Mitigations/Mitigation/0 string, how to mitigates

.../Prerequisites/Prerequisite/0 string, attack prerequisite

.../Related_Attack_Patterns/Related_Attack_Pattern/0
- .../@CAPEC_ID int, related CAPEC ID
- .../@Nature string, relationship type: ChildOf, CanPrecede, PeerOf

.../Related_Weaknesses/Related_Weakness/0
- .../@CWE_ID int, related CWE ID, e.g. “276”

.../Resources_Required/Resource/0 string

.../Typical_Severity string, e.g. “Low”, “Medium”, “High”, “Very High”

3.5. MITRE ENGAGE

MITRE Engage is a framework for planning and discussing adversary engagement operations. It defines.

The main Engage Github repository seems to be from 2022. The JSON files posted in the Data/json directory define the concepts and relationships in this framework. It seems there is no explicit schema document.

The JSON file structure differs from ATT&CK and all others. Concept IDs are given as dictionary keys in some files or as values to “id” keys. Relationships in the concept *_detailed.json files are specified with adjacency lists given as properties. Additionally, relationships between Engage concepts and mappings to ATT&CK concepts are listed in several .json files, as seen below. Information related to relationships is repeated several times. The overall design seems very ad-hoc.

The repository includes mappings (relationships) between ATT&CK Techniques and Tactics that are relevant for Engage, defined in file attack_tactics_techniques.json. At this time I could not determine if these relationships are reproduced from those defined in the ATT&CK dataset.

The Engage schema pulls in from ATT&CK these concepts:

- attack_technique, as course-of-action and attack-pattern
- attack_tactic, as x-mitre-tactic

and includes all relevant details in its own implicit classes (e.g. attack_technique) included in file activity_details.json. Vulnerabilities are **not** mapped to CVE or CWE.

Key concepts and relationships:

The concepts are:

Concept	**File name**	**Property**	**Comment or Ex.**
Goal	goal_details.json	(id, key in dict)	
		name	Expose
		description	
		long_description	
		type	Engagement
Activity	activity_details.json	(id, key in dict)	EAC0001
		name	API Monitoring
		type	Engagement
		description	
		long_description	
		goals (rel.)	list[goal id]
		vulnerabilities (rel.)	list[vuln]
		attack_techniques (rel)	list[attck tec]
		attack_tactics (rel)	list[attck tac]
		approaches (rel)	list[appr id]
		references (rel)	list[ref]
Approach	approach_details.jsonl	(id, key in dict)	
		type	"Engagement"
		name	
		description	
		long_description	
		goals (rel)	list[goal id]
		activities (rel)	list[activ.id]
vulnerability	activity_details.json	id	EAV0001
		eav	description
attack_technique	activity_details.json	id	ATT&CK, course-of-action technique ID
		attack_tactics (rel)	list[str]
		name	ATT&CK name (~)
attack_tactic	activity_details.json (very thin)	id	ATT&CK ID, x-mitre-tactic
		name	string
reference	activity_details.json and in references.json:	id	e.g. REF0007
		title	
		url	
		activity_id (rel)	

Relationships are captured by attributes in the details files and _mappings files, as follows.

Some relationship are

Relationship	File name	Comment
Activity - Goal	activity_details.jsonl	
Activity - Approach	activity_details.jsonl	
Activity - vulnerability	activity_details.jsonl	
Activity - attack_technique	activity_details.jsonl	indirect to ATT&CK technique
Activity - attack_tactic	activity_details.jsonl	
Activity - reference	activity_details.jsonl	
Approach - Goal	approach_details.jsonl	
Approach - Activity	approach_details.jsonl	
Goal - Approach	goal_approach_mappings.jsonl and goal_details.jsonl	
attack_technique - ATTACK:technique	activity_details.jsonl, attack_mapping.jsonl	
attack_technique - vulnerability	attack_mapping.jsonl	
attack_technique - Activity	attack_mapping.jsonl	
attack_technique - ATTACK:x-mitre-tactic	attack_tactics_techniques.jsonl	
attack_tactic - ATTACK:x-mitre-tactic	activity_details.jsonl	
vulnerability - attack_tactic	activity_details.jsonl vul. as key in act. dct	
reference - Activity	activity_details.json, references.jsonl	

3.6. Neo4J CTI Graph Database Schema

The CTI graph DB will need more refinement in these directions:

A. Subgraphs related to a CTI concept (e.g. CWE) are captured by a single node in the neo4j DB that refers to a dictionary. Explicit navigable relationships are missing. For example, several CWE records refer to the same Detection_Method. CWEs “Use of Inherently Dangerous Function” and “Creation of chroot Jail Without Changing Working Directory” and “ Detection_Method” refer to Detection_Method DM-14 “Automated Static Analysis”.
B. Some CTI concepts and their relationships are missing, such as *x-mitre-data-source*, *x-mitre-data-component.*
C. ATT&CK *revoked_by* relationships are missing.
D. Neo4J nodes must include all the properties needed by Cypher queries, including last-modified date, name, external id.

NOTE: A CTI domain expert involved in template formulation must weigh in and coordinate with the CTI DB team on the necessary CTI graph schema.

The information below was obtained on 9/5/2025 from the Athena CTI Neo4J server.

The graph is shown below, in Figure 2:

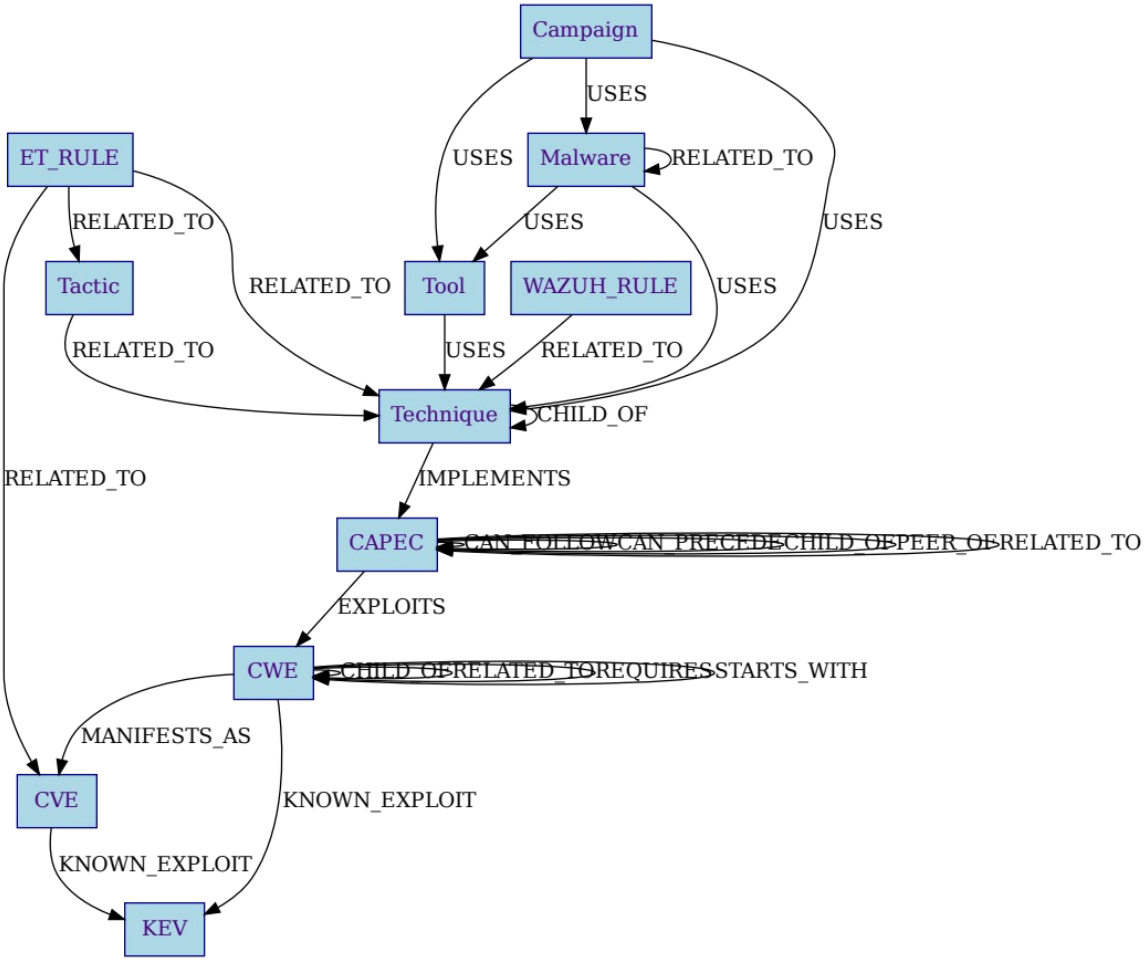

Figure 2: ASG neo4j CTI database extracted using Cypher queries.

The nodes and properties of the Neo4J CTI **graph model** are listed next:
(File schema_graph_ASG-CTI.txt)

```
Nodes:
Node                    Property                                Comments
CAPEC
                        capec_id
                        stix_id
CVE
                        cve_id
                        stix_id
CWE
                        cwe_id
                        stix_id
Campaign
                        mitre_id
                        stix_id
ET_RULE
                        framework
                        sid
                        stix_id
```

```
KEV
                        cve_id
                        stix_id
Malware
                        mitre_id
                        stix_id
Tactic
                        mitre_id
                        shortname
                        stix_id
Technique
                        is_subtechnique
                        kill_chain_phase_name
                        mitre_id
                        stix_id
Tool
                        mitre_id
                        stix_id
WAZUH_RULE
                        framework
                        level
                        rule_id
                        stix_id
```

Edges:

```
Edge Type
CAPEC CAN_FOLLOW CAPEC
CAPEC CAN_PRECEDE CAPEC
CAPEC CHILD_OF CAPEC
CAPEC EXPLOITS CWE
CAPEC PEER_OF CAPEC
CAPEC RELATED_TO CAPEC
CVE KNOWN_EXPLOIT KEV
CWE CHILD_OF CWE
CWE KNOWN_EXPLOIT KEV
CWE MANIFESTS_AS CVE
CWE RELATED_TO CWE
CWE REQUIRES CWE
CWE STARTS_WITH CWE
Campaign USES Malware
Campaign USES Technique
Campaign USES Tool
ET_RULE RELATED_TO CVE
ET_RULE RELATED_TO Tactic
ET_RULE RELATED_TO Technique
Malware RELATED_TO Malware
Malware USES Technique
Malware USES Tool
Tactic RELATED_TO Technique
Technique CHILD_OF Technique
Technique IMPLEMENTS CAPEC
Tool USES Technique
WAZUH_RULE RELATED_TO Technique
```

Total: 11 nodes, 27 edges

Note that edges have no properties.

It seems that more node and edge properties are accessible using a PostgresSQL database. However, IFT data generation requires the ability to query the graph with Cypher using node and edge properties not available yet in the neo4j graph database.

4. Design

4.1. Template Language Grammar

This is an EBNF grammar written for the Lark Python parser module.

```
template:     topfield+

topfield:     qfield | TEXTSECTION | invsection | listsection

qfield:       "{" vardef? scope? cnameseq subscript? "}"

vardef:       CNAME ":"
scope:        CNAME "#"
cnameseq:     CNAME ("." CNAME)*
subscript:    "[" subscr_exp "]"

subscr_exp:   QUANTIFIER | (QUANTIFIER SUBSCR_OP QSTRING)

invsection:   "<*" (INVCONTENT | qfield)* "*>"

listsection:  "[" (TEXTSECTION | qfield)* "]"

INVCONTENT:   /(?:[^*{]|\*(?!>))+/
QFTEXT:       /(?:[^}])+/
TEXTSECTION:  /(?:[^{<[\]]|<(?!\*))+/


DIGIT:        "0".."9"
CNAMELETTER:  "a".."z" | "A".."Z" | "_"
CNAMECHAR:    CNAMELETTER | "-"
CNAME:        CNAMELETTER (CNAMECHAR|DIGIT)*

QUANTIFIER:   QUANTIF_ANY | QUANTIF_ALL
QUANTIF_ANY:  "?"
QUANTIF_ALL:  "*"
SUBSCR_OP:    "!=" | "="
QSTRING:      "\"" QSTRING_DQ "\"" | "'" QSTRING_SQ "'"
QSTRING_DQ:   /(?:[^"\\]|\\.)*/
QSTRING_SQ:   /(?:[^'\\]|\\.)*/
```

The Lark module builds a parse tree that is further processed with a Lark.Transformer object twice:

1. to generate the Cypher query.
2. to parse the result set from neo4j and to construct the final generated template text.

Building a prototype parser is ongoing work.

4.2. Parser Design

[TBD]